

Corso di Laurea in Ingegneria e Scienze Informatiche

Fancy Title

Tesi di laurea in:
NOME DEL CORSO DEL RELATORE

Relatore

Prof. Supervisor Here

Candidato

Candidate Name

Abstract

Opzionale. Poche righe al massimo.

Contents

Abstract	iii
1 Introduzione	1
1.1 Contesto e motivazione	1
1.2 Obiettivi del progetto	1
1.3 Struttura della tesi	1
2 Background	3
2.1 Sviluppo di applicazioni full stack monolinguaggio	3
2.1.1 Panoramica delle soluzioni esistenti e loro limiti	4
2.1.2 Il portabilità e integrazione con librerie native	4
2.2 Kotlin e framework multitarget come risposta al gap	4
2.2.1 Kotlin Multiplatform	4
2.2.2 Interoperabilità con C tramite Cinterop	4
2.3 Lo standard FMI e il formato FMU	4
2.3.1 Functional Mock-up Interface (FMI 2.0)	4
2.3.2 Struttura di un file FMU e modalità operative	4
3 Analisi	5
3.1 Requisiti	5
3.2 Modello del dominio	5
3.2.1 Vincoli	5
4 Design	7
4.1 Architettura	7
4.2 Design di dettaglio	7
4.2.1 Interfacciamento con librerie native	7
4.2.2 Backend multiplatforma	7
4.2.3 Frontend multiplatforma	7

5	Implementazione	9
5.1	Binding C/Kotlin e configurazione del linker per piattaforma	9
5.2	Gestione del ciclo di vita dell'FMU	9
5.3	Estrazione dei metadati e gestione delle variabili	9
5.4	Esecuzione della simulazione	9
5.5	Ricompilazione degli FMU su macOS ARM64	9
5.6	Strumenti di processo: Gradle, CI/CD e pipeline multiplatforma .	9
6	Valutazione	11
6.1	Test unitari e di integrazione	11
6.2	Copertura multiplatforma nella CI	11
7	Conclusioni e Sviluppi Futuri	13
7.1	Limitazioni	13
7.2	Sviluppi futuri	13
7.3	Conclusioni	13
		15
	Bibliography	15

List of Figures

LIST OF FIGURES

List of Listings

LIST OF LISTINGS

Chapter 1

Introduzione

1.1 Contesto e motivazione

1.2 Obiettivi del progetto

1.3 Struttura della tesi

Chapter 2

Background

2.1 Sviluppo di applicazioni full stack monolinguaggio

Prima di addentrarci nella descrizione del background del progetto è importante definire in maniera corretta cosa sono le applicazioni **full stack** e le differenze che emergono tra lo sviluppo di esse e quello di applicazioni più tradizionali. Le componenti delle applicazioni vengono divise in due macrocategorie

- **Frontend:** in questa sezione dell'applicativo sono gestiti tutti gli aspetti inerenti all'interfaccia utente: sia quelli legati all'aspetto grafico che quelli legati alla logica di interazione con essa.
- **Backend:** in questa sezione invece sono gestiti tutti gli aspetti legati all'esecuzione delle funzionalità dell'applicazione, inclusa l'elaborazione dei dati, la loro memorizzazione e la comunicazione con altre applicazioni se necessario.

Le applicazioni *full stack* dunque, comprendono entrambe le componenti. Lo sviluppo di questo tipo di applicazioni può includere l'utilizzo di molteplici linguaggi in base alle necessità, impiegando linguaggi progettati per lo sviluppo del backend e altri per il frontend. Tuttavia questa distinzione al giorno d'oggi viene gradualmente smussata, infatti diversi linguaggi inizialmente progettati per

l'utilizzo in una sola area, vengono poi estesi anche all'altra. L'esempio più rilevante e dominante al giorno d'oggi è **JavaScript**, il quale inizialmente nasce come linguaggio per i browser, quindi dedicato al *frontend*, per poi evolversi oltre i confini del browser permettendone l'esecuzione anche sui server. Questo permette dunque la creazione di applicativi *full stack* monolinguaggio, una tendenza sempre più diffusa. Tra i vantaggi che essa comporta vi sono l'utilizzo di un solo linguaggio per tutto lo stack, il quale porta a una minore curva di apprendimento e una maggiore efficienza per quanto riguarda l'impiego di programmatori. Uno dei più grandi limiti dello sviluppo *full stack* è l'interazione con librerie e API native, infatti linguaggi come JavaScript sono noti per avere meccanismi limitati o comunque ostici per interagire con il codice nativo. In seguito verranno discusse in dettaglio le soluzioni esistenti per la programmazione di tale tipo di applicativi e i loro limiti, per poi analizzare il problema della portabilità e dell'integrazione con le librerie native.

2.1.1 Panoramica delle soluzioni esistenti e loro limiti

2.1.2 Il portabilità e integrazione con librerie native

2.2 Kotlin e framework multitarget come risposta al gap

2.2.1 Kotlin Multiplatform

2.2.2 Interoperabilità con C tramite Cinterop

2.3 Lo standard FMI e il formato FMU

2.3.1 Functional Mock-up Interface (FMI 2.0)

2.3.2 Struttura di un file FMU e modalità operative

Chapter 3

Analisi

3.1 Requisiti

3.2 Modello del dominio

3.2.1 Vincoli

Chapter 4

Design

4.1 Architettura

4.2 Design di dettaglio

4.2.1 Interfacciamento con librerie native

4.2.2 Backend multiplatforma

4.2.3 Frontend multiplatforma

Chapter 5

Implementazione

- 5.1 Binding C/Kotlin e configurazione del linker per piattaforma
- 5.2 Gestione del ciclo di vita dell'FMU
- 5.3 Estrazione dei metadati e gestione delle variabili
- 5.4 Esecuzione della simulazione
- 5.5 Ricompilazione degli FMU su macOS ARM64
- 5.6 Strumenti di processo: Gradle, CI/CD e pipeline multiplatforma

Chapter 6

Valutazione

6.1 Test unitari e di integrazione

6.2 Copertura multiplatforma nella CI

Chapter 7

Conclusioni e Sviluppi Futuri

7.1 Limitazioni

7.2 Sviluppi futuri

7.3 Conclusioni

Bibliography